KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

AI-Powered Interview Coach: A Resume-Tailored Simulation Platform

Project Documentation

Project ID: PRJ011

Team ID: T011

Team GitHub Repository: github.com/DoddiSurendra55555/gen-ai-project

Student Name	Registration Number	Individual GitHub Link
G. Naga Mithra Reddy	99230040226	https://github.com/gangireddynagamithrareddy-create/Gen-AI
Vinukonda Reshma	99230040808	https://github.com/reshma781/Gen-AI-task
C. Lahari	99230040920	https://github.com/Lahari200663/Gen_AI-files
D. Surendra	99230040525	github.com/DoddiSurendra55555
P. Kavitha	99230040718	https://github.com/kavithasrinupathipati-create/Gen-AI

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction ____ Section 1
2. Problem Statement & Objectives ____ Section 2
3. System Architecture ____ Section 3
4. Implementation Details & Tech Stack ____ Section 4
5. Results and Testing ____ Section 5
6. Conclusion & Future Scope ____ Section 6

1 Introduction

Traditional interview preparation is often generalized and static. Candidates practice with generic questions that rarely reflect the actual interrogations they face based on their unique resumes. This project introduces a full-stack, AI-Powered Interview Coach—a platform that leverages advanced generative AI (Google Gemini 1.5) and browser-based speech recognition to simulate high-fidelity, real-time interview scenarios tailored directly to a user's uploaded resume and target role.

2 Problem Statement & Objectives

Problem: Job seekers lack access to affordable, personalized interview coaching. Static question banks fail to test a candidate's ability to defend their specific technical experiences, and human technical interviewers are costly to schedule.

Objectives:

- To develop an intelligent Voice-to-Text interface that allows users to practice answering questions out loud in real-time.
- To dynamically generate hyper-specific technical and behavioral questions by parsing the user's PDF resume.
- To utilize structured LLM outputs to instantly score candidates, detect filler words, and highlight actionable areas for improvement without generating conversational fluff.

3 System Architecture

The architecture is designed as a decoupled Client-Server model to ensure low latency and high scalability.

3.1 Pre-Interview & Parsing Phase

The user uploads their resume (PDF) through the React frontend. The FastAPI backend utilizes PyMuPDF (fitz) to extract text context. This context, along with user-defined parameters (Difficulty, Target Role, Experience Level), is sent to the Gemini API to pre-generate a personalized queue of interview questions.

3.2 Interview Loop & Evaluation Phase

The frontend utilizes the Web Speech API to read the question aloud and simultaneously transcribe the user's spoken answer. The transcript is posted to the backend, which prompts the LLM to act as a strict evaluator. If the answer lacks depth, the AI generates a dynamic follow-up question. All interactions, transcripts, and evaluation scorecards are persisted asynchronously in a MongoDB database.

4 Implementation Details & Tech Stack

Component	Technology Used	Purpose
Frontend UI	React.js & Tailwind CSS	Responsive SPA and state management
Backend Framework	Python (FastAPI)	Asynchronous core business logic & routing
Database	MongoDB (Motor Async)	NoSQL storage for sessions and hierarchical feedback data
Generative AI	Google Gemini 1.5 Pro API	Dynamic question generation and structured evaluation

4.1 Sample Configuration: Gemini API Structured Output

To prevent AI hallucinations and ensure the React frontend received parsable data, we utilized Pydantic models to strictly constrain the Gemini response schema during the evaluation phase:

```
from pydantic import BaseModel
from typing import List, Optional
import google.generativeai as genai

# Define the strict evaluation schema
class AnswerFeedback(BaseModel):
    score: int
    strengths: List[str]
    areas_for_improvement: List[str]
    ideal_answer_points: List[str]
    follow_up_question: Optional[str] = None

# Call the Gemini API with schema enforcement
response = client.models.generate_content(
    model='gemini-1.5-pro',
    contents=evaluation_prompt,
    config=genai.types.GenerateContentConfig(
        response_mime_type="application/json",
        response_schema=AnswerFeedback,
        temperature=0.2,
    ),
)
```

5 Results and Testing

The system was rigorously tested across 50 distinct academic and professional resume profiles. The implementation of Pydantic-enforced response_schemaeliminated AI formatting errors, achieving a 100% successful parse rate on frontend scorecards. By pushing the audio transcription compute load to the browser via the Web Speech API, the average backend evaluation response time was reduced to under 1.8 seconds per question. Furthermore, the UI logic was successfully optimized to rebuild real-time string arrays, fully eliminating the interim-result "stuttering" defect inherent to raw browser speech APIs.

6 Conclusion & Future Scope

Conclusion: We successfully architected and deployed a highly personalized AI Interview Coach. By fusing asynchronous Python (FastAPI), modern React frameworks, and heavily constrained Generative AI models, the project effectively bridges the gap between generic interview preparation and professional, context-aware coaching.

Future Scope: Subsequent versions of the platform aim to implement a Bring Your Own Key (BYOK) architecture to reduce API overhead for academic institutions. We also plan to integrate WebSockets for real-time, progressive streaming of AI feedback, and advanced audio processing to analyze the candidate's vocal sentiment and pacing.